# UNIDAD 4
# CREACIÓN DE BASES DE DATOS EN MYSQL (DDL)

## FICHA TÉCNICA DE LA UNIDAD DIDÁCTICA

| | |
|---|---|
| **Unidad Didáctica** | Unidad Didáctica 4: Creación de bases de datos en MySQL (Lenguaje de Definición de Datos - DDL) |
| **Resultados de Aprendizaje** | **RA3:** Realiza el diseño físico de bases de datos utilizando asistentes, herramientas gráficas y el lenguaje de definición de datos. |
| **Criterios de Evaluación** | CE3.a, CE3.b, CE3.c, CE3.d, CE3.e, CE3.f, CE3.g, CE3.h, CE3.i |
| **Tiempo Estimado** | 18 Horas (Planificado para 3 semanas de sesiones teórico-prácticas) |

## ÍNDICE DE CONTENIDOS

1. INTRODUCCIÓN Y CONTEXTUALIZACIÓN

2. EL LENGUAJE DDL DE SQL

3. MANIPULACIÓN DE BASES DE DATOS EN MYSQL

```
3.1
Creación de Bases de Datos: Parámetros del Sistema
3.1.1
Conceptos de Codificación de Caracteres: Estándar Unicode
3.1.2
Gestión de utf8 (alias utf8mb3) vs utf8mb4 en Entornos de
Producción
3.1.3
Definición de CHARACTER SET y Colaciones (COLLATE)
3.1.4
Impacto del Cotejamiento en la Ordenación Física de Tablas
3.2
Modificación de Bases de Datos (ALTER DATABASE)
```

3.3 Eliminación Segura
(DROP DATABASE)

## 4. MANIPULACIÓN DE TABLAS

4.1 Sentencia de
Creación (CREATE TABLE)
4.1.1
Restricciones de Integridad y Atributos de Columna
4.1.2
Integridad Referencial: Opciones en Claves Ajenas (FOREIGN
KEY)
4.1.3 Configuración
de Opciones de Tabla
4.2 Modificación
Estructural (ALTER TABLE)
4.2.1 Redefinición de
Atributos (MODIFY)
4.2.2
Renombrado y Cambio Estructurado (CHANGE)
4.2.3
Gestión de Valores por Defecto (ALTER ... SET/DROP DEFAULT)
4.2.4
Adición y Eliminación de Atributos (ADD / DROP COLUMN)
4.3 Eliminación de
Tablas (DROP TABLE)

## 5. TIPOS DE DATOS EN MYSQL

5.1 Clasificación y Representación en Memoria de Enteros

5.1.1 Atributo ZEROFILL y el mito de INT (11)

5.1.2 Tipos Especiales (BIT, BOOL, SERIAL)

5.2 Datos en Punto Flotante (Valores Aproximados y Precisión Computacional)

5.3 Datos en Punto Fijo (Valores Exactos para Datos Financieros)

5.4 Fechas y Tiempo

5.5 Cadenas de Caracteres

5.6 Datos Binarios

5.7 Tipos Estructurados Especiales (ENUM, SET)

5.8 Tipos No Relacionales: Almacenamiento y Operaciones JSON

5.9 Configuración y Gestión de sql_mode

# 1. INTRODUCCIÓN Y CONTEXTUALIZACIÓN

El objetivo primordial de esta unidad es capacitar al futuro administrador de sistemas en la implementación, modificación y optimización de las estructuras lógicas de datos sobre el motor de base de datos MySQL. Se busca no solo que el alumno domine la sintaxis estándar de SQL-DDL, sino que comprenda las implicaciones a nivel de rendimiento en disco, r

Esta unidad se conecta directamente con los módulos profesionales de **Administración de Sistemas Operativos en Red (ASO)** e **Implantación de Aplicaciones Web (IAW)**. El aprovisionamiento de infraestructuras para aplicaciones como CMS (WordPress, Drupal) o plataformas empresariales requiere que el administrador configure de forma óptima el servidor de bases de datos relacionales, modificando variables globales de ejecución (sql_mode), definiendo esquemas eficientes y aislando el almacenamiento físico de la base de datos para garantizar la alta disponibilidad y la integridad referencial de los servicios corporativos.

# 2. EL LENGUAJE DDL DE SQL

El DDL (data definition language) es la especificación de SQL encargada de definir las estructuras que almacenan los datos del sistema. Las sentencias de esta especificación se ejecutan de forma síncrona en el diccionario de datos de MySQL e impactan directamente sobre el sistema de archivos del servidor (por ejemplo, ficheros .ibd en motores InnoDB).

> 🔗 **Documentación Oficial**: Para consultar la lista de todas las sentencias de definición de datos y la gestión del diccionario de datos de MySQL, accede a MySQL Manual - Data Definition Statements.

Estructura jerárquica de comandos SQL de definición

- CREATE: Instancia nuevas estructuras (DATABASE, TABLE, INDEX, VIEW, USER)

- ALTER: Modifica metadatos y estructuras físicas existentes
- DROP: Elimina permanentemente estructuras físicas y sus contenidos
- SHOW: Consulta el estado del diccionario de datos y variables de sesión/global

Sentencias de utilidad

- USE <nombre_bd> Selecciona el esquema activo en el hilo de conexión actual.
- DESCRIBE <nombre_tabla>; o DESC <nombre_tabla>;: Recupera los metadatos de las columnas de una tabla para validar el esquema.

# 3. MANIPULACIÓN DE BASES DE DATOS EN MYSQL

## 3.1 Creación de Bases de Datos: Parámetros del Sistema

En MySQL, las palabras clave DATABASE y SCHEMA son sinónimos absolutos. La directiva IF NOT EXISTS evita que se interrumpa la ejecución de scripts automatizados de despliegue (como Ansible o scripts de Shell) si el esquema ya se encuentra en el servidor.

```
-- Creación básica (Aplica el juego de caracteres del servidor por
defecto)
CREATE DATABASE IF NOT EXISTS sysadmin_prod;
-- Creación óptima recomendada para compatibilidad internacional
CREATE DATABASE IF NOT EXISTS sysadmin_prod
CHARACTER SET utf8mb4
COLLATE utf8mb4_unicode_ci;
```

### 3.1.1 Conceptos de Codificación de Caracteres: Estándar Unicode

Unicode define un conjunto universal de caracteres mapeado numéricamente. La codificación UTF-8 representa este conjunto usando bloques dinámicos de bytes:

- Caracteres ASCII clásicos: 1 byte.
- Bloques alfabéticos adicionales: 2 bytes.

- Resto del BMP (Basic Multilingual Plane): 3 bytes.
- Caracteres complementarios (Emojis, caracteres ideográficos): 4 bytes.

### 3.1.2 Gestión de utf8 (alias utf8mb3) vs utf8mb4 en Entornos de Producción

En las versiones heredadas de MySQL, la codificación denominada utf8 utiliza únicamente un máximo de 3 bytes por carácter. Esto imposibilita almacenar de forma nativa caracteres fuera del BMP, como la simbología emoji o caracteres asiáticos complejos. Para evitar fallas de inserción y truncado de strings en entornos corporativos modernos, es mandatorio utilizar el juego de caracteres utf8mb4, el cual sí soporta hasta 4 bytes.

## 3.1.3 Definición de CHARACTER SET y Colaciones (COLLATE)

- **CHARACTER SET:** Indica qué codificación binaria se usa para mapear caracteres a bytes en disco.
- **COLLATE:** Determina las reglas lógicas para la ordenación física y las comparaciones de igualdad entre strings.
  - _cs (Case Sensitive): Diferencia entre mayúsculas y minúsculas (Ej: a distinto de A).
  - _ci (Case Insensitive): No diferencia mayúsculas de minúsculas (Ej: a igua A).
  - _bin (Binary): Las comparaciones se ejecutan directamente sobre el valor de la representación numérica binaria en memoria del carácter.

> 🔗 **Documentación Oficial**: Para profundizar sobre cómo MySQL maneja los juegos de caracteres dinámicos y la compatibilidad con Unicode completo, revisa MySQL Manual - Character Sets, Collations, Unicode.

```
-- Inspección del juego de caracteres y colaciones disponibles en el
motor
SHOW CHARACTER SET;
SHOW COLLATION LIKE "utf8mb4%";
-- Consulta del juego de caracteres y la colación configurados en la
base de datos actual
USE sysadmin_prod;
SELECT @@character_set_database, @@collation_database;
```

### 3.1.4 Impacto del Cotejamiento en la Ordenación Física de Tablas

El cotejamiento afecta de forma radical a consultas ordenadas (ORDER BY). A continuación se ejemplifica cómo se almacena y ordena el diccionario de datos en base al cotejamiento asignado a la columna o tabla:

```
-- Creación de tabla de prueba con codificación clásica latina
CREATE TABLE tabla_cotejo (
cadena CHAR(3) CHARACTER SET latin1
);
INSERT INTO tabla_cotejo (cadena) VALUES ("AAA"), ("bbb"), ("aaa"),
("BBB");
-- 1. Ordenación con Cotejo Sensible a Mayúsculas (_cs)
-- Salida esperada: aaa -> AAA -> bbb -> BBB
SELECT cadena FROM tabla_cotejo ORDER BY cadena COLLATE
latin1_general_cs;
-- 2. Ordenación con Cotejo Insensible a Mayúsculas (_ci)
-- Salida esperada: AAA/aaa se tratan como iguales a nivel de
ordenación, igual con BBB/bbb
SELECT cadena FROM tabla_cotejo ORDER BY cadena COLLATE
latin1_swedish_ci;
-- 3. Ordenación Binaria (_bin)
-- Ordena rigurosamente según el valor del byte ASCII (Mayúsculas
tienen códigos menores que minúsculas)
-- Salida esperada: AAA -> BBB -> aaa -> bbb
SELECT cadena FROM tabla_cotejo ORDER BY cadena COLLATE
latin1_bin;
```

## 3.2 Modificación de Bases de Datos (ALTER DATABASE)

Permite alterar los parámetros estructurales por defecto del esquema sin necesidad de recrear los objetos internos.

```
-- Cambio de codificación en caliente para dar soporte a Unicode
completo
ALTER DATABASE sysadmin_prod CHARACTER SET utf8mb4 COLLATE
utf8mb4_unicode_ci;
```

## 3.3 Eliminación Segura (DROP DATABASE)

Elimina físicamente el directorio que contiene los ficheros de la base de datos y todas sus tablas. Debe usarse con extrema precaución en scripts de automatización.

```
-- Eliminación controlada de un esquema de datos
DROP DATABASE IF EXISTS sysadmin_prod;
```

# 4. MANIPULACIÓN DE TABLAS

## 4.1 Sentencia de Creación (CREATE TABLE)

A continuación se detalla la sintaxis estructurada para instanciar tablas físicas aplicando restricciones del modelo relacional:

🔗 **Documentación Oficial:** La sintaxis formal, opciones avanzadas y limitaciones de la sentencia de creación se detallan en **MySQL Manual - CREATE TABLE Statement**.

```
-- Creación de la base de datos para el despliegue del esquema
CREATE DATABASE IF NOT EXISTS logistica_empresa CHARSET utf8mb4;
USE logistica_empresa;
-- Creación de Tabla Principal (Entidad Fuerte)
CREATE TABLE categoria (
id INT UNSIGNED AUTO_INCREMENT,
nombre VARCHAR(100) NOT NULL,
-- Declaración de Clave Primaria en la especificación de tabla
CONSTRAINT pk_categoria PRIMARY KEY (id)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;
-- Creación de Tabla Relacionada (Entidad Débil con Clave Ajena y
restricción CHECK)
CREATE TABLE pieza (
id INT UNSIGNED AUTO_INCREMENT,
nombre VARCHAR(100) NOT NULL,
color VARCHAR(50) NOT NULL,
precio DECIMAL(7,2) NOT NULL,
id_categoria INT UNSIGNED NOT NULL,
-- Clave Primaria
CONSTRAINT pk_pieza PRIMARY KEY (id),
-- Restricción de integridad Check (Soportada nativamente desde MySQL
8.0)
CONSTRAINT chk_precio_positivo CHECK (precio > 0),
-- Definición de la clave ajena
```

```
CONSTRAINT fk_pieza_categoria FOREIGN KEY (id_categoria)
REFERENCES categoria(id)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;
```

### 4.1.1 Restricciones de Integridad y Atributos de Columna

- NOT NULL / NULL: Indica obligatoriedad de valor.
- DEFAULT <valor>: Asigna un valor por defecto si se omite en la sentencia INSERT.
- AUTO_INCREMENT: Autonumérico incremental de enteros. Requiere que la columna sea clave primaria o índice único.
- UNIQUE: Impide la duplicidad de valores en la columna.
- PRIMARY KEY: Restringe el campo como identificador único de la tupla (implica indexación física por árbol B+ B-tree y restricción NOT NULL).

### 4.1.2 Integridad Referencial: Opciones en Claves Ajenas (FOREIGN KEY)

Las cláusulas ON DELETE y ON UPDATE controlan el comportamiento del sistema ante la manipulación de datos maestros en cascada:

- RESTRICT: Deniega el borrado o actualización en la tabla padre si existen registros huérfanos asociados en la tabla hija. Es la opción por defecto.
- CASCADE: Sincroniza de forma recursiva los cambios. Si se borra la categoría con id = 1, se eliminarán de forma automatizada todas las piezas pertenecientes a dicha categoría.
- SET NULL: Si se modifica o elimina el registro padre, actualiza las claves foráneas de la tabla hija a valor NULL (requiere que la columna no tenga restricción NOT NULL).
- NO ACTION: Equivale semánticamente a RESTRICT en motores MySQL.

🔗 **Documentación Oficial:** **Para comprender a fondo el funcionamiento, restricciones e impacto de rendimiento de las restricciones de clave ajena, visita MySQL Manual - FOREIGN KEY Constraints.**

```
-- Ejemplo de configuración relacional con Cascade para gestión
automatizada
CREATE TABLE pieza_cascade (
id INT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
nombre VARCHAR(100) NOT NULL,
```

```
id_categoria INT UNSIGNED NOT NULL,
CONSTRAINT fk_pieza_cat_cascade FOREIGN KEY (id_categoria)
REFERENCES categoria(id)
ON DELETE CASCADE
ON UPDATE CASCADE
) ENGINE=InnoDB;
```

### 4.1.3 Configuración de Opciones de Tabla

Al final de la sentencia de creación se definen opciones físicas de almacenamiento:

- ENGINE=InnoDB: Motor transaccional ACID nativo de MySQL con bloqueo a nivel de fila e integridad referencial real.
- AUTO_INCREMENT = <valor_inicial>: Configura el puntero de inserción autonumérico.

```
-- Configuración avanzada de almacenamiento
CREATE TABLE usuarios_logs (
id INT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
accion VARCHAR(255) NOT NULL
) ENGINE=InnoDB AUTO_INCREMENT=1000 DEFAULT CHARSET=utf8mb4;
```

## 4.2 Modificación Estructural (ALTER TABLE)

Esencial para aplicar parches de migración de datos en entornos de producción sin pérdida de registros existentes.

🔗 **Documentación Oficial: Consulta la sintaxis completa y las operaciones en caliente (*Online DDL*) que se pueden realizar en el motor InnoDB mediante MySQL Manual - ALTER TABLE Statement.**

### 4.2.1 Redefinición de Atributos (MODIFY)

Modifica el tipo de dato o las restricciones asociadas a una columna existente, manteniendo el nombre de esta.

```
-- Cambiar la columna "nombre" para aceptar más caracteres y hacerla
obligatoria
ALTER TABLE categoria MODIFY nombre VARCHAR(150) NOT NULL;
```

### 4.2.2 Renombrado y Cambio Estructurado (CHANGE)

Permite renombrar una columna física además de redefinir su tipo de dato de forma simultánea.

```
-- Modifica el nombre de la columna "nombre" a
"nombre_categoria"
ALTER TABLE categoria CHANGE nombre nombre_categoria VARCHAR(100) NOT
NULL;
```

### 4.2.3 Gestión de Valores por Defecto (ALTER ... SET/DROP DEFAULT)

Permite la modificación del metadato por defecto sin modificar el tipo de columna.

```
-- Añadir rol por defecto al registrar un usuario nuevo
ALTER TABLE usuarios_logs ADD COLUMN rol ENUM("SysAdmin", "Operador")
NOT NULL;
ALTER TABLE usuarios_logs ALTER rol SET DEFAULT "Operador";
-- Eliminar valor por defecto
ALTER TABLE usuarios_logs ALTER rol DROP DEFAULT;
```

### 4.2.4 Adición y Eliminación de Atributos (ADD / DROP COLUMN)

Define las reglas físicas de ubicación de nuevas columnas dentro de la estructura de la tupla física en disco.

```
-- Añade una columna al final de la tabla
ALTER TABLE usuarios_logs ADD COLUMN timestamp_registro TIMESTAMP NOT
NULL;
-- Añade columnas controlando la posición exacta
ALTER TABLE usuarios_logs ADD COLUMN ip_origen VARCHAR(45) NOT NULL
AFTER accion;
ALTER TABLE usuarios_logs ADD COLUMN id_interno INT FIRST;
-- Eliminar columna física y sus datos asociados permanentemente
ALTER TABLE usuarios_logs DROP COLUMN id_interno;
```

## 4.3 Eliminación de Tablas (DROP TABLE)

Elimina la estructura de metadatos, los índices físicos y los datos contenidos de la tabla del disco.

```
-- Borrado unitario y múltiple controlado
DROP TABLE IF EXISTS pieza;
DROP TABLE IF EXISTS pieza_cascade, usuarios_logs;
```

# 5. TIPOS DE DATOS EN MYSQL

Elegir de forma óptima el tipo de dato previene el desperdicio de almacenamiento en disco y la saturación del pool de memoria (Buffer Pool de InnoDB).

🔗 **Documentación Oficial:** Para consultar la referencia técnica de todos los tipos de datos disponibles, rangos y almacenamiento físico en bytes, revisa MySQL Manual - Data Types.

## 5.1 Clasificación y Representación en Memoria de Enteros

| Tipo de Dato | Tamaño en Disco | Rango (Signed) | Rango (Unsigned) |
|---|---|---|---|
| TINYINT | 1 Byte | [-128,127] | [0,255] |
| SMALLINT | 2 Byte | [-32768, 32767] | [0,65535] |
| MEDIUMINT | 3 Byte | [-8388608, 8388607] | [0, 16777215] |
| INT / INTEGER | 4 Byte | $-2^{31}$ a $2^{31}-1$ | 0 a $2^{32}-1$ |
| BIGINT | 8 Byte | $-2^{63}$ a $2^{63}-1$ | 0 a $2^{64}-1$ |

### 5.1.1 Atributo ZEROFILL y el mito de INT (11)

La declaración de un tipo de dato como INT(11) no restringe el almacenamiento ni el valor del entero a 11 dígitos numéricos; es exclusivamente un metadato de formato de salida visual. Cuando se utiliza conjuntamente con el modificador ZEROFILL, MySQL rellenará con ceros a la izquierda de forma visual en las terminales que soporten este atributo hasta alcanzar el tamaño especificado.

```
-- Ejemplo práctico de visualización
CREATE TABLE demo_zerofill (
numero INT(4) ZEROFILL
);
```

```
INSERT INTO demo_zerofill VALUES (5);
-- Al hacer SELECT, la salida mostrada será: "0005"
```

### 5.1.2 Tipos Especiales (BIT, BOOL, SERIAL)

- BIT(M): Almacena valores a nivel de bits, donde 1 a $2^6$ (64).
- BOOL / BOOLEAN: Alias del motor para TINYINT(1). El valor 0 equivale a FALSE y cualquier entero se asume como TRUE.
- SERIAL: Alias que equivale de forma explícita a: BIGINT UNSIGNED NOT NULL AUTO_INCREMENT UNIQUE.

## 5.2 Datos en Punto Flotante (Valores Aproximados y Precisión Computacional)

Los tipos FLOAT y DOUBLE representan aproximaciones de números reales en punto flotante bajo el estándar IEEE-754. **Nunca** deben usarse en sistemas transaccionales monetarios debido a los redondeos e imprecisiones intrínsecas de las operaciones binarias internas de la CPU.

- FLOAT(M, D): Simple precisión (4 bytes), con precisión de rango de aprox. $\pm 1.1 \times 10^{38}$ a aprox. $\pm 3.7540 \times 10^{38}$.
- DOUBLE(M, D): Doble precisión (8 Bytes), con rango de aprox. $\pm 2{,}23 \times 10^{-308}$ a aprox. $\pm 1.79 \times 10^{308}$.

## 5.3 Datos en Punto Fijo (Valores Exactos para Datos Financieros)

Los tipos DECIMAL y NUMERIC son equivalentes. Almacenan los números de forma exacta (conversión a cadenas binarias empaquetadas por bloques de 9 dígitos en MySQL), preservando la precisión monetaria de las operaciones.

- DECIMAL(M, D): M representa la precisión total de dígitos de la que se compone el número entero (rango de 1 a 65), D representa el número de cifras decimales (rango de 0 a 30).

```
-- Definición idónea para transacciones bancarias o e-commerce
CREATE TABLE transaccion (
```

```
id INT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
importe DECIMAL(10,2) NOT NULL -- Permite hasta 8 dígitos enteros y 2
decimales
);
```

## 5.4 Fechas y Tiempo

| Tipo de Dato | Tamaño | Formato por Defecto | Rango Soportado |
|---|---|---|---|
| DATE | 3 Bytes | YYYY-MM-DD | 1000-01-01 a 9999-12-31 |
| DATETIME | 8 Bytes | YYYY-MM-DD HH:MM:SS | 1000-01-01 00:00:00 a 9999-12-31 23:59:59 |
| TIMESTAMP | 4 Bytes | YYYY-MM-DD HH:MM:SS | 1970-01-01 00:00:01 UTC a 2038-01-19 03:14:07 UTC |
| TIME | 3 Bytes | HH:MM:SS | -838:59:59 a 838:59:59 |
| YEAR | 1 Byte | YYYY | 1901 a 2155 |

⚠️ **El problema del año 2038:** Los campos TIMESTAMP almacenan los datos internamente como el número de segundos transcurridos desde la época Unix (1970-01-01 00:00:00 UTC). Al ser un entero de 32 bits con signo, el valor máximo alcanzable expirará el martes 19 de enero de 2038 a las 03:14:07 UTC. A partir de ese instante el valor desbordará provocando errores críticos. Para nuevos diseños lógicos que almacenen fechas lejanas en el futuro, es obligatorio planificar la migración estructural a DATETIME (8 bytes).

🔗 **Documentación Oficial:** Para conocer las ventajas y diferencias operativas entre DATETIME y TIMESTAMP, lee MySQL Manual - Date and Time Types.

## 5.5 Cadenas de Caracteres

- CHAR(M): Cadena de longitud fija, donde 0 a 255. Si el string introducido es de longitud inferior a M, el motor rellena físicamente el espacio sobrante con espacios en disco. Es ideal para hashes (MD5, SHA-256) o identificadores uniformes.
- VARCHAR(M): Cadena de longitud variable, donde 0 a 65535. Utiliza 1 o 2 Bytes adicionales como prefijo para almacenar la longitud lógica en bytes de la cadena ingresada. Ideal para

nombres, correos y descripciones.

- TEXT: Para almacenamiento de texto plano a gran escala. No almacena los datos directamente en la página del índice clustered si exceden ciertos límites, sino en páginas externas de desbordamiento.
    - TEXT (Soporta hasta $2^{16}$Bytes = 64 KB.
    - MEDIUMTEXT (Soporta hasta $2^{24}$ Bytes = 16 MB.
    - LONGTEXT (Soporta hasta $2^{32}$Bytes = 4 GB

## 5.6 Datos Binarios

La familia BLOB (Binary Large Object) se comporta igual que TEXT pero para bytes planos (archivos binarios, imágenes, cifrados binarios). En arquitectura e ingeniería de sistemas, **no se recomienda** almacenar archivos directamente en el motor de base de datos debido a que degrada el rendimiento de lectura del disco y satura el sistema de copias de seguridad (mysqldump). Es mejor guardar el archivo en el sistema de almacenamiento de archivos del servidor y registrar únicamente la ruta de acceso en una columna VARCHAR.

## 5.7 Tipos Estructurados Especiales (ENUM, SET)

- ENUM('val1', 'val2', ...): Permite almacenar únicamente un único valor de una lista restringida predefinida de strings (almacenado de forma interna de forma eficiente como un entero). Máximo de 65535 valores posibles.
- SET('val1', 'val2', ...): Permite almacenar de cero a múltiples valores simultáneos de la lista de strings permitidos. Máximo de 64 valores permitidos en la lista.

## 5.8 Tipos No Relacionales: Almacenamiento y Operaciones JSON

Nativo desde MySQL 5.7 y optimizado en 8.0, permite validar sintácticamente documentos JSON estructurados guardándolos de forma binaria transparente en disco.

🔗 **Documentación Oficial:** Conoce todas las funciones y ventajas operativas de almacenar documentos no relacionales de forma nativa en [MySQL Manual - JSON Data Type].

```
-- Creación de tabla para almacenamiento no relacional
estructurado
CREATE TABLE auditoria_config (
id INT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
metadatos JSON NOT NULL
);
-- Inserción de documentos JSON válidos
INSERT INTO auditoria_config (metadatos) VALUES
('{"servidor": "web_prod_01", "ip": "192.168.10.50", "servicios":
["http", "https"]}');
-- Extracción de propiedades JSON estructuradas mediante alias y
operadores directos ->
SELECT metadatos->"$.servidor" AS servidor,
metadatos->"$.servicios[0]" AS puerto_principal
FROM auditoria_config;
```

## 5.9 Configuración y Gestión de sql_mode

La variable del sistema sql_mode define el nivel de rigor del motor a la hora de procesar excepciones, desbordamiento de enteros y conversión implícita de tipos.

🔗 **Documentación Oficial:** Para comprender los efectos del modo estricto en la prevención de la inserción de datos corruptos o fuera de rango en producción, consulta MySQL Manual - Server SQL Modes.

```
-- Consulta del valor actual de sql_mode del servidor
SELECT @@GLOBAL.sql_mode;
SELECT @@SESSION.sql_mode;
-- Configuración del modo estricto estandarizado (SysAdmin de
Producción)
-- El modo STRICT_TRANS_TABLES o TRADITIONAL deniega inserciones que
provoquen truncado de datos o desbordamiento (lanza ERROR)
SET SESSION sql_mode = "STRICT_TRANS_TABLES,NO_ZERO_IN_DATE,
NO_ZERO_DATE,ERROR_FOR_DIVISION_BY_ZERO";
-- Demostración de desbordamiento (Comportamiento con sql_mode
estricto vs no estricto)
CREATE TABLE test_overflow (
valor TINYINT UNSIGNED
);
-- Si sql_mode es estricto (STRICT_TRANS_TABLES): Lanza ERROR 1264 e
impide la inserción
-- Si sql_mode es vacío (" "): Inserta el valor máximo representable
```

```
(255) y muestra un WARNING
INSERT INTO test_overflow VALUES (300);
```